

# Alertmanager Silence Management Tool

Automated tool for creating and managing Alertmanager silences during database maintenance operations at GitLab.

The script is located at [https://gitlab.com/gitlab-com/gl-infra/db-migration/-/blob/master/alert-management/alert\\_management.py](https://gitlab.com/gitlab-com/gl-infra/db-migration/-/blob/master/alert-management/alert_management.py)

**Compatible with macOS and Linux | Uses 1Password for secure credential management**

---

## Table of Contents

- Overview
  - Prerequisites
  - Installation
    - macOS Setup
    - Linux Setup (Debian/Ubuntu)
    - Linux Setup (RHEL/CentOS/Fedora)
  - Quick Start
  - Usage Guide
    - Authentication
    - Creating Silences
    - Listing Silences
    - Getting Silence Details
    - Expiring Silences
  - Real-World Examples from Runbooks
  - Command Reference
  - Maintenance Workflow
  - Troubleshooting
  - Advanced Usage
  - Support
- 

## Overview

This tool provides a command-line interface for managing Alertmanager silences during database maintenance windows. It eliminates the need for manual silence creation via the web UI and enables automation of alert management in maintenance runbooks.

### Key Features:

- Secure credential management via 1Password
- Support for hours and minutes duration
- Advanced filtering with type and tier matchers

- List, create, view, and expire silences
- Cross-platform (macOS and Linux)
- Integration-ready for Ansible and CI/CD

---

## Prerequisites

Before installing, ensure you have:

| Requirement             | Check Command                  | Minimum Version         |
|-------------------------|--------------------------------|-------------------------|
| <b>Python 3</b>         | <code>python3 --version</code> | 3.8+                    |
| <b>pip3</b>             | <code>pip3 --version</code>    | Latest                  |
| <b>1Password CLI</b>    | <code>op --version</code>      | 2.0+                    |
| <b>1Password Access</b> | Ask ops team                   | Production vault access |

---

## Installation

### macOS Setup

```
# 1. Install 1Password CLI using Homebrew
brew install --cask 1password-cli

# 2. Clone the repository (or download the files)
cd /path/to/alertmanager-tools

# 3. Install Python dependencies
pip3 install -r requirements.txt

# 4. Sign in to 1Password
op signin

# 5. Verify your setup
chmod +x check_setup.sh
./check_setup.sh
```

### Expected output:

All checks passed! Your setup is complete.

---

### Linux Setup (Debian/Ubuntu)

```
# 1. Install 1Password CLI
curl -sS https://downloads.1password.com/linux/keys/1password.asc | \
```

```

sudo gpg --dearmor --output /usr/share/keyrings/1password-archive-keyring.gpg
echo 'deb [arch=amd64 signed-by=/usr/share/keyrings/1password-archive-keyring.gpg] https://c
sudo tee /etc/apt/sources.list.d/1password.list

sudo mkdir -p /etc/debsig/policies/AC2D62742012EA22/
curl -sS https://downloads.1password.com/linux/debian/debsig/1password.pol | \
sudo tee /etc/debsig/policies/AC2D62742012EA22/1password.pol

sudo mkdir -p /usr/share/debsig/keyrings/AC2D62742012EA22
curl -sS https://downloads.1password.com/linux/keys/1password.asc | \
sudo gpg --dearmor --output /usr/share/debsig/keyrings/AC2D62742012EA22/debsig.gpg

sudo apt update && sudo apt install 1password-cli

# 2. Install Python dependencies
pip3 install -r requirements.txt

# 3. Sign in to 1Password
op signin

# 4. Verify setup
chmod +x check_setup.sh
./check_setup.sh

```

---

## Linux Setup (RHEL/CentOS/Fedora)

```

# 1. Install 1Password CLI
sudo rpm --import https://downloads.1password.com/linux/keys/1password.asc

sudo sh -c 'echo -e "[1password]\nname=1Password Stable Channel\nbaseurl=https://downloads.1

sudo dnf install 1password-cli

# 2. Install Python dependencies
pip3 install -r requirements.txt

# 3. Sign in to 1Password
op signin

# 4. Verify setup
chmod +x check_setup.sh
./check_setup.sh

```

---

## Quick Start

After installation, here's a 30-second test:

```
# 1. Sign in to 1Password (if not already signed in)
op signin

# 2. List currently active silences
python3 alert_management.py list --state active

# 3. Create a test silence (15 minutes)
python3 alert_management.py create \
  --alert-pattern "TestAlert" \
  --duration-minutes 15 \
  --comment "Testing the alert management tool" \
  --environment ops

# 4. Verify it was created
python3 alert_management.py list --state active

# 5. Expire the test silence (using the ID from step 4)
python3 alert_management.py expire --silence-id <silence-id-from-step-4>
```

---

## Usage Guide

### Authentication

Before using the tool, authenticate with 1Password:

```
# Sign in (interactive - will prompt for your 1Password password)
op signin
```

```
# Your session remains active for a period of time
# Re-run this command if you see authentication errors
```

For CI/CD environments, use a 1Password service account token:

```
export OP_SERVICE_ACCOUNT_TOKEN="ops_your_token_here"
python3 alert_management.py create ...
```

---

### Creating Silences

#### Basic Syntax

```
python3 alert_management.py create \
  --alert-pattern "<alert-name-or-pattern>" \
```

```

--duration-hours <hours> \
--comment "<description>" \
--environment <env>

```

**Duration Options** You can specify duration in **hours** OR **minutes** (not both):

```

# Using hours (for longer maintenance)
--duration-hours 4

```

```

# Using minutes (for quick tasks)
--duration-minutes 30

```

**Optional Matchers** Add precision to your silences with optional matchers:

```

# Type matcher (e.g., specific database cluster)
--type "gprd-patroni-main-v16"

```

```

# Tier matcher (e.g., database tier)
--tier "db"

```

```

# Both together
--type "patroni-main" --tier "db"

```

### Complete Example

```

python3 alert_management.py create \
--alert-pattern "PostgresDown|PostgresReplicationLag" \
--duration-hours 4 \
--comment "PostgreSQL 17 upgrade on patroni-main cluster" \
--environment gprd \
--type "patroni-main" \
--tier "db"

```

### Output:

```

==> Creating silence...
Alert Pattern: PostgresDown|PostgresReplicationLag
Environment: gprd
Type: patroni-main
Tier: db
Duration: 4h (240 minutes)
Start: 2026-01-20T10:30:00Z
End: 2026-01-20T14:30:00Z
Comment: PostgreSQL 17 upgrade on patroni-main cluster

```

Silence created successfully!

Silence ID: abc12345-6789-def0-1234-56789abcdef0

View: <https://alerts.gitlab.net/#/silences/abc12345-6789-def0-1234-56789abcdef0>

To expire this silence, run:

```
python alert_management.py expire --silence-id abc12345-6789-def0-1234-56789abcdef0
```

---

## Listing Silences

### List All Active Silences

```
python3 alert_management.py list --state active
```

#### Output:

Found 3 silence(s):

```
[ACTIVE] abc12345-6789-def0-1234-56789abcdef0
Alert Pattern: PostgresDown|PostgresReplicationLag
Environment: gprd
Type: patroni-main
Created by: alertmanager-silence-auto@gitlab-ops.iam.gserviceaccount.com
Started: 2026-01-20T10:30:00Z
Ends: 2026-01-20T14:30:00Z
Comment: PostgreSQL 17 upgrade on patroni-main cluster
```

### List All Silences (Active + Expired)

```
python3 alert_management.py list
```

### Filter by Creator

*# Show only automation-created silences*

```
python3 alert_management.py list --created-by "alertmanager-silence-auto"
```

*# Show only silences created by a specific person*

```
python3 alert_management.py list --created-by "Vamshi"
```

---

## Getting Silence Details

View detailed information about a specific silence:

```
python3 alert_management.py get --silence-id abc12345-6789-def0-1234-56789abcdef0
```

#### Output:

Silence Details:

```
ID: abc12345-6789-def0-1234-56789abcdef0
State: ACTIVE
```

```
Created by: alertmanager-silence-auto@gitlab-ops.iam.gserviceaccount.com
Started: 2026-01-20T10:30:00Z
Ends: 2026-01-20T14:30:00Z
Updated: 2026-01-20T10:30:00Z
Comment: PostgreSQL 17 upgrade on patroni-main cluster
```

Matchers:

- alertname: PostgresDown|PostgresReplicationLag (regex)
  - environment: gprd
  - type: patroni-main
  - tier: db
- 

## Expiring Silences

End a maintenance window early by expiring the silence:

```
python3 alert_management.py expire --silence-id abc12345-6789-def0-1234-56789abcdef0
```

**Output:**

```
==> Expiring silence abc12345-6789-def0-1234-56789abcdef0...
Silence expired successfully!
Silence ID: abc12345-6789-def0-1234-56789abcdef0
```

---

## Real-World Examples from Runbooks

These examples are taken directly from GitLab database maintenance runbooks:

### Example 1: WALGBaseBackup Silence for patroni-main-v16 (122 hours)

**Scenario:** PostgreSQL backup maintenance on patroni-main-v16 cluster

```
python3 alert_management.py create \
--alert-pattern "WALGBaseBackupFailed|walgBaseBackupDelayed" \
--duration-hours 122 \
--comment "WALGBaseBackup alerts silence for patroni-main-v16 maintenance" \
--environment gprd \
--type "gprd-patroni-main-v16"
```

**When to use:** During base backup system maintenance or when rebuilding WAL-G backup infrastructure

---

### Example 2: WALGBaseBackup Silence for patroni-main-v17 (76 hours)

**Scenario:** Backup system maintenance on PostgreSQL 17 cluster

```
python3 alert_management.py create \  
  --alert-pattern "WALGBaseBackupFailed|walgBaseBackupDelayed" \  
  --duration-hours 76 \  
  --comment "WALGBaseBackup alerts silence for patroni-main-v17 maintenance" \  
  --environment gprd \  
  --type "gprd-patroni-main-v17"
```

**When to use:** PostgreSQL 17 cluster backup maintenance window

---

### Example 3: PostgresSplitBrain Silence (96 hours / 4 days)

**Scenario:** Patroni cluster reconfiguration that may trigger split-brain detection

```
python3 alert_management.py create \  
  --alert-pattern "PostgresSplitBrain" \  
  --duration-hours 96 \  
  --comment "PostgresSplitBrain alert silence for patroni-main maintenance" \  
  --environment gprd \  
  --type "patroni-main"
```

**When to use:**

- Major Patroni cluster reconfigurations
  - Network maintenance affecting cluster consensus
  - Planned failover testing
- 

### Example 4: ChefClientDisabled Silence (48 hours / 2 days)

**Scenario:** Infrastructure maintenance requiring Chef client to be disabled

```
python3 alert_management.py create \  
  --alert-pattern "ChefClientDisabled" \  
  --duration-hours 48 \  
  --comment "ChefClientDisabled alert silence for infrastructure maintenance" \  
  --environment gprd \  
  --type "patroni-main"
```

**When to use:**

- Infrastructure updates requiring Chef to be disabled
- Migration or configuration changes
- System rebuilds

---

### Example 5: Complete Database Cluster Maintenance

**Scenario:** Full maintenance window with multiple alert types, type, and tier matchers

```
python3 alert_management.py create \  
  --alert-pattern "PostgresDown|PostgresReplicationLag|PostgresReplicationStopped|PatroniClu\  
  --duration-hours 4 \  
  --comment "Full database cluster maintenance - PostgreSQL upgrade and failover testing" \  
  --environment gprd \  
  --type "patroni-main" \  
  --tier "db"
```

**When to use:**

- PostgreSQL major version upgrades
  - Comprehensive cluster testing
  - Major infrastructure changes
- 

### Example 6: Quick Chef Client Update (30 minutes)

**Scenario:** Quick Chef client update across database nodes

```
python3 alert_management.py create \  
  --alert-pattern "ChefClientError" \  
  --duration-minutes 30 \  
  --comment "Quick Chef client update on database nodes" \  
  --environment gstg
```

**When to use:**

- Quick Chef client updates
  - Configuration deployments
  - Short maintenance tasks
- 

## Command Reference

**create** - Create a new silence

**Required Arguments:**

- **--alert-pattern** - Alert name or regex pattern (e.g., "PostgresDown|PostgresReplicationLag")
- **--duration-hours** OR **--duration-minutes** - Duration of the silence
- **--comment** - Description of the maintenance

**Optional Arguments:**

- `--environment` - Environment (default: `gprd`). Options: `gprd`, `gstg`, `ops`
  - `--type` - Type matcher (e.g., `"patroni-main"`, `"gprd-patroni-main-v16"`)
  - `--tier` - Tier matcher (e.g., `"db"`, `"sv"`, `"inf"`)
  - `--op-item` - 1Password item name (default: `"Alertmanager Service Account Key"`)
  - `--op-vault` - 1Password vault name (default: `"Production"`)
- 

## **list - List silences**

### **Optional Arguments:**

- `--state` - Filter by state: `active` or `expired`
- `--created-by` - Filter by creator name (partial match)

### **Examples:**

```
# All active silences
python3 alert_management.py list --state active

# All silences (including expired)
python3 alert_management.py list

# Silences created by automation
python3 alert_management.py list --created-by "alertmanager-silence-auto"
```

---

## **get - Get silence details**

### **Required Arguments:**

- `--silence-id` - The silence ID

### **Example:**

```
python3 alert_management.py get --silence-id abc12345-6789-def0-1234-56789abcdef0
```

---

## **expire - Expire a silence**

### **Required Arguments:**

- `--silence-id` - The silence ID to expire

### **Example:**

```
python3 alert_management.py expire --silence-id abc12345-6789-def0-1234-56789abcdef0
```

---

## Maintenance Workflow

Here's a recommended workflow for database maintenance:

### 1. Pre-Maintenance Planning

```
# Check for any existing active silences
python3 alert_management.py list --state active

# Review what alerts might fire during your work
# Consult runbooks or past maintenance records
```

### 2. Create Silence Before Starting Work

```
# Create a silence with appropriate duration
python3 alert_management.py create \
  --alert-pattern "PostgresDown|PostgresReplicationLag" \
  --duration-hours 4 \
  --comment "PostgreSQL upgrade on patroni-main - Ticket: DB-12345" \
  --environment gprd \
  --type "patroni-main"

# Save the silence ID from the output
# Silence ID: abc12345-6789-def0-1234-56789abcdef0
```

### 3. Perform Maintenance

Carry out your database maintenance tasks...

### 4. Post-Maintenance Cleanup

```
# If finished early, expire the silence
python3 alert_management.py expire --silence-id abc12345-6789-def0-1234-56789abcdef0

# Or let it expire automatically at the scheduled time

# Verify no unexpected silences remain
python3 alert_management.py list --state active
```

### 5. Document in Runbook

Add the silence creation command to your runbook for future reference.

## Troubleshooting

### “Not signed in to 1Password”

**Problem:** `op signin` session has expired

**Solution:**

`op signin`

---

### “Error fetching service account key from 1Password”

**Problem:** No access to the Production vault or item doesn't exist

**Solutions:**

1. Verify you have access:  

```
op vault list
op document list --vault "Production"
```
  2. Request access from ops team if needed
  3. Verify the item name is correct (default: “Alertmanager Service Account Key”)
- 

### “Python 3 not found” (Linux)

**Problem:** Python 3 not installed

**Solution:**

```
# Debian/Ubuntu
sudo apt update && sudo apt install python3 python3-pip

# RHEL/CentOS/Fedora
sudo dnf install python3 python3-pip
```

---

### “No module named ‘google’” or “No module named ‘requests’”

**Problem:** Python dependencies not installed

**Solution:**

```
pip3 install -r requirements.txt
```

---

## “Permission denied” when running check\_setup.sh

**Problem:** Script not executable

**Solution:**

```
chmod +x check_setup.sh
./check_setup.sh
```

---

## Silence not created or “Invalid IAP credentials”

**Problem:** Service account not properly configured or IAP permissions missing

**Solutions:**

1. Verify the service account exists and has proper IAM roles
  2. Contact ops team to verify IAP configuration
  3. Check if the service account key in 1Password is current
- 

## Advanced Usage

### Common Alert Patterns

| Pattern                                            | Use Case                                     |
|----------------------------------------------------|----------------------------------------------|
| PostgresDown                                       | Database down alerts                         |
| PostgresReplicationLag\ PostgresReplicationStopped | Replication issues                           |
| PatroniClusterNotHealthy                           | Patroni cluster problems                     |
| ChefClientError\ ChefClientDisabled                | Chef configuration management                |
| WALGBaseBackupFailed\ walgBaseBackupDelayed        | Backup system issues                         |
| PostgresSplitBrain                                 | Split-brain scenarios                        |
| Postgres.*\ Patroni.*                              | All Postgres/Patroni alerts (use carefully!) |

### Common Type Values

| Type                  | Description                         |
|-----------------------|-------------------------------------|
| patroni-main          | Main Patroni cluster                |
| gprd-patroni-main-v16 | PostgreSQL 16 cluster in production |
| gprd-patroni-main-v17 | PostgreSQL 17 cluster in production |
| patroni-ci            | CI/CD database cluster              |
| blackbox              | Blackbox monitoring                 |

## Common Tier Values

| Tier | Description         |
|------|---------------------|
| db   | Database tier       |
| sv   | Services tier       |
| inf  | Infrastructure tier |

## Using in Ansible Playbooks

```
- name: Create Alertmanager silence
  command: >
    python3 alert_management.py create
    --alert-pattern "{{ alert_pattern }}"
    --duration-hours {{ duration_hours }}
    --comment "{{ comment }}"
    --environment {{ environment }}
    --type "{{ type_matcher }}"
  register: silence_result

- name: Extract silence ID
  set_fact:
    silence_id: "{{ silence_result.stdout | regex_search('Silence ID: ([a-f0-9-]+)', '\\1') }}"

- name: Perform database maintenance
  # ... your maintenance tasks ...

- name: Expire silence
  command: >
    python3 alert_management.py expire
    --silence-id {{ silence_id }}
```

---

## Platform-Specific Notes

### macOS

- Use `python3` and `pip3` explicitly
- 1Password CLI installed via Homebrew (`brew install --cask 1password-cli`)
- Shell: Typically `zsh` (default since macOS Catalina)

### Linux

- Ensure you have `curl`, `gpg`, and package manager access
- May need `sudo` for 1Password CLI installation

- Use `python3` and `pip3` explicitly
  - Shell: Typically `bash`
- 

## Support

### Getting Help

#### Documentation:

- This README file
- Built-in help: `python3 alert_management.py --help`
- Command-specific help: `python3 alert_management.py create --help`

#### Internal Resources:

- **Issue Tracker:** `gitlab-com/gl-infra/data-access/dbo/dbo-issue-tracker`
- **Slack Channel:** `#database-sre`
- **Runbooks:** Check your team's runbook repository for more examples

## Reporting Issues

When reporting an issue, please include:

1. The full command you ran
2. The complete error message
3. Your OS (macOS/Linux) and version
4. Output of `./check_setup.sh`

## Contributing

Improvements and bug fixes are welcome! Please follow your team's contribution guidelines.

---

## Quick Reference Card

Save this for quick access:

```
# Sign in
op signin

# Create silence (4 hours)
python3 alert_management.py create \
  --alert-pattern "AlertName" \
  --duration-hours 4 \
  --comment "Maintenance description" \
```

```
--environment gprd

# List active
python3 alert_management.py list --state active

# Expire
python3 alert_management.py expire --silence-id <id>

# Help
python3 alert_management.py --help
```

---

**Last Updated:** January 2026 **Maintained By:** Database Team **Version:** 1.0.0